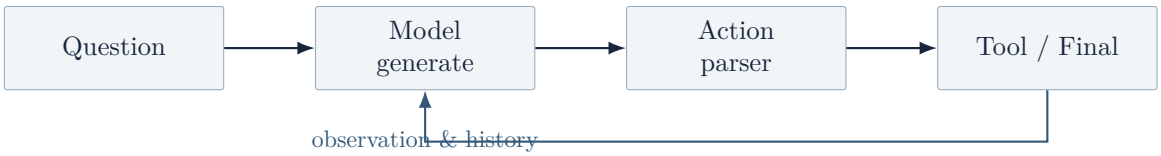
Building a ReAct Agent from Scratch

A CS336-Style Programming Assignment

Action Protocol • Tool Execution
Agent Loop • Multi-Hop QA Evaluation



Version	4.1 (English edition)
Date	23 October 2026
Recommended stack	Python 3.11+ / uv / pytest
Models	DeepSeek API or local Ollama (Qwen2:7B)
Benchmark	HotpotQA distractor (closed-candidate setting)

Abstract

In this assignment, you will **implement the entire agent inference system outside the language model**: an action parser, a tool registry and executor, retrieval and document lookup, message-history management, a budgeted ReAct loop, model-provider adapters, execution traces, and offline evaluation on HotpotQA. Each section first introduces the relevant concepts, then specifies function contracts, implementation steps, edge cases, and acceptance tests. Complete and test one module before moving to the next. You **do not need to train a language model, use a GPU, or adopt an agent framework**. Following the pedagogical style of Stanford CS336 Assignment 1, this handout specifies the functionality in detail without supplying implementation solutions. It is an independently written study assignment, not official Stanford course material.

Contents

1 Assignment overview: what you will build **3**
- 1.1 Expected behavior and scope . . . 3
- 1.2 What you implement, what you run, and what you may use . . . 3
- 1.3 Suggested repository layout: separate tests from implementation . . . 3
- 1.4 Grading and progressive milestones . . . 5

2 Foundations: models, tools, and environments (6 points) **5**

3 Action protocol: making model output parseable (12 points) **6**
- 3.1 Exactly two permitted action formats . . . 6
- 3.2 Public interface and a suggested decomposition . . . 7

4 Retrieval: start with three documents (12 points) **8**
- 4.1 Fixed toy environment and evidence indices . . . 8
- 4.2 Minimal retrieval algorithm: token-set overlap . . . 8

5 Tool executor: controlled action execution (10 points) **9**

6 Messages, prompting, and the ReAct loop (20 points) **10**
- 6.1 Message format and visible history . . . 10
- 6.2 State-transition table and important boundary cases . . . 11
- 6.3 Isolate controller bugs with a scripted model . . . 12

7 Provider adapters: DeepSeek and Ollama (10 points) **13**

8 Trace, replay, and experimental integrity (6 points) **14**

9 HotpotQA: from toy documents to real data (6 points) **15**
- 9.1 Prevent gold-label leakage . . . 16

10 Metrics and batch evaluation (12 points) **17**
- 10.1 Fixed splits: smoke, then development, then evaluation . . . 18
- 10.2 Batch execution and the official prediction format . . . 18

11 Controls, ablation, and error analysis (6 points) 19

12 Submission checklist and debugging guide 20

12.1 Final submission 20

12.2 Acceptance checklist 20

12.3 Where to investigate common failures first 21

13 Optional extensions (outside the base 100 points) 21

A Interface reference: all public functions 22

B Test matrix: what to test before writing the full agent 23

C Reference materials 23

1 Assignment overview: what you will build

1.1 Expected behavior and scope

Given a natural-language question, the model generates *exactly one machine-readable action* at each decision step: invoke `search`, invoke `lookup`, or submit `final`. Your program parses and validates that action, executes authorized tools subject to budgets, appends the resulting observation to the message history, and asks the model for the next decision. Once the run terminates, it must return the answer, evidence citations, termination status, and full execution record.

Knowledge: What exactly does ReAct mean in this assignment?

The central idea of ReAct is the alternating *reason / act / observe* process: choose the next step using currently visible information, interact with an external environment, observe the result, and reconsider the next step. The original paper's trajectories can include explicitly written Thoughts. To support multiple backends, this assignment uses **structured JSON actions plus visible observations**. A short, optional decision note is allowed, but **private chains of thought are neither required nor collected**. You are building a lightweight tool-using agent inspired by ReAct, not reproducing every prompting template in the original paper. This is inference, not reinforcement-learning training.

1.2 What you implement, what you run, and what you may use

What you implement. (i) A strict JSON action parser; (ii) deterministic document retrieval; (iii) a safe tool executor; (iv) prompting, history, and an agent loop; (v) entry points for two model backends; (vi) trace logging in JSONL; (vii) a HotpotQA loader with per-question tools; and (viii) EM/F1 metrics, batch execution, and experimental comparisons.

What you run. First, pass offline unit tests without network access. Next, run a scripted model on a three-document toy corpus. Then run at least one real end-to-end example with DeepSeek *or* Ollama. Finally, measure actual scores on fixed HotpotQA subsets. You must generate and report all empirical results yourself; illustrative examples in this handout are not benchmark measurements.

What you may use. The Python standard library, `pytest`, `httpx`, the `openai` SDK (*only as an LLM transport*), `python-dotenv`, `uv`, and the official HotpotQA evaluator. You **may not** replace your parser, executor, history, loop, or evaluation with LangChain, LangGraph, CrewAI, AutoGen, or similar agent frameworks. Do not evaluate or execute model-generated text using `eval` or `exec`; do not copy a third-party ReAct implementation.

Guidance on AI tools. You may use AI to clarify concepts, inspect library APIs, or understand error messages. To preserve the educational purpose of implementing everything yourself, write the core functions independently rather than asking an AI coding agent to implement them end to end. Disclose any substantive external assistance in your report.

1.3 Suggested repository layout: separate tests from implementation

```
react-from-scratch/
  pyproject.toml                # you create / edit
  .env.example                  # no real secrets
  README.md
  adapters.py                   # thin public API forwarding only
  react_agent/
    __init__.py
    protocol.py                 # you write
    retrieval.py                # you write
```

```
    tools.py                    # you write
    prompting.py                # you write
    agent.py                    # you write
    providers.py                # you write
    tracing.py                  # you write
    dataset.py                  # you write
    metrics.py                  # you write
    evaluation.py               # you write
  tests/                        # create tests; NEVER modify to hide bugs
    conftest.py
    test_protocol.py
    test_retrieval.py
    test_tools.py
    test_loop.py
    test_metrics.py
    test_dataset.py
  scripts/
    check_backend.py
    make_splits.py
    evaluate.py
  data/                         # downloaded dataset; not in code zip
  reports/
    writeup.md
    splits/
    runs/
```

These are **suggested filenames**, not source files bundled with the PDF. The assignment provides specifications rather than an implementation repository: create the code and test files yourself. Your `adapters.py` should forward the specified public interfaces to your implementations and contain no substantive business logic. Test individual functions first rather than trying to build the entire application in a single step.

1.4 Grading and progressive milestones

Stage	Deliverable	Points	Evidence before proceeding
M0	Concepts and development environment	6	Explain model / tools / controller
M1	JSON action protocol	12	Positive and negative parser tests
M2	Candidate-document retrieval	12	Stable search and lookup
M3	Safe tool execution	10	Controlled invalid-tool behavior
M4	Messages and ReAct loop	20	Complete toy problem
M5	DeepSeek / Ollama backend	10	Real end-to-end request
M6	Traces and resume	6	Reconstruct decisions and calls
M7	HotpotQA data isolation	6	Preserve citations; prevent leakage
M8	Metrics and batch evaluation	12	Recompute scores on fixed IDs
M9	Baselines, ablations, analysis	6	Interpretable measured results
	Total	**100**	

Milestone: Where to start (no API key required)

Create the project with `uvinit` and add `pytest`. Complete only the two valid and five invalid JSON examples in §3. Before your first parser test succeeds, you do not need HotpotQA, a local model, or access to the internet.

2 Foundations: models, tools, and environments (6 points)

Knowledge: An LLM does not directly execute Python functions

A **model** maps message histories to text: $M : \mathcal{H} \to \mathcal{Y}$. A **tool** is a controlled callable $T(a)$ rather than an arbitrary function the model can execute. A **controller** maintains state between the model and tools, validating actions, enforcing permissions and budgets, handling exceptions, extending history, and deciding when to stop. The **environment** consists of the candidate documents that can be retrieved for the current question.

Let h_t be the message history visible to the model at step t, a_t a decoded and validated action, and o_t the tool observation:

$$a_t \sim \pi_\theta(\cdot \mid h_t), \qquad o_t = T(a_t), \qquad h_{t+1} = h_t \oplus \text{assistant}(a_t) \oplus \text{user}(o_t).$$

Here $\oplus$ denotes appending messages in order. The parameters θ belong to a **pretrained model** and are not updated in this assignment. The history h_t is a textual context, not the network's

hidden state. Writing a trace to disk does not automatically expose the recorded information to the model.

Problem (foundations, 6 points)

In `reports/writeup.md`, answer these questions (2 points each; approximately 100–180 words per answer).

1. **Component boundaries.** Why is a system-prompt instruction such as "call at most six tools" insufficient to guarantee termination? Which component must check the budget?
2. **Trajectory.** How is h_t similar to a trajectory in reinforcement learning? Why does this assignment not need a reward model, policy gradient, or backpropagation?
3. **Threat model.** If a webpage, retrieval result, or tool error says "ignore the rules above," why must this text be treated as *data* rather than a new instruction?

Deliverable. Three short responses and a message-flow diagram that includes at least one tool call followed by a final answer (a hand-drawn diagram is acceptable in the report).

3 Action protocol: making model output parseable (12 points)

Knowledge: Syntactically valid does not imply executable

Raw model output is untrusted: it may be prose, fenced JSON, a top-level array, a missing field, or a request for a nonexistent tool. Validate three separate layers: **syntax** (the whole string parses as JSON), **structure** (the decoded object satisfies the action contract), and **execution** (the tool is registered, its arguments are accepted, and budget remains). This section implements the first two layers only; execution comes later.

3.1 Exactly two permitted action formats

Every `model.generate(messages)` call must return **exactly one JSON object**. Leading and trailing whitespace is allowed; code fences, explanatory paragraphs, and additional objects are not. An optional `note` gives a short, visible explanation of the action choice. It is not scored, must be a string if present, and is not a request for a detailed hidden chain of thought.

```
{"type":"tool_call", "tool":"search",
 "args":{"query":"Ada Lovelace father", "k":3},
 "note":"Find the first relevant article"}

{"type":"final", "answer":"London",
 "evidence":[["Ada Lovelace",1],["Lord Byron",1]]}
```

Field	Applies to	Exact requirement
`type`	Both	Required string: `tool_call` or `final`
`tool`	Tool call	Nonempty string after stripping surrounding whitespace
`args`	Tool call	JSON object; an empty object is valid, but an array is not
`answer`	Final	String; an empty string is allowed and will be scored accordingly
`evidence`	Final	Optional; default `[]`; otherwise a list of sentence citations
`note`	Either	Optional short string, not a detailed private reasoning trace

Each evidence entry must be `[title,sent_id]`, where `title` is a nonempty string and `sent_id` a **nonnegative, zero-based integer**. In Python, `bool` is a subclass of `int`, so both `True` and `False` must be rejected as sentence indices. The parser does **not** check whether titles exist or tools are registered: both depend on the runtime environment.

3.2 Public interface and a suggested decomposition

`parse_action(text:str)->dict`

Purpose: Convert the complete assistant text into a validated action dictionary with a fixed structure.

Contract: Input is a string; success returns at least `type` and the fields required by that action. Default absent `evidence` to `[]`. Irrelevant extra fields may be ignored.

Failure/edge: Raise `ValueError` for invalid JSON or malformed action structure; never execute tools.

First apply `json.loads`, verify the top-level container is a dictionary, branch on `type`, and validate the required fields. Do not start by developing an elaborate JSON Schema framework.

Problem (protocol-valid, 5 points)

Implement the valid-input path in small increments: **(a)** final with an answer; **(b)** final with evidence; **(c)** tool call with a tool and arguments; **(d)** optional note. Add one unit test for each new behavior.

```
input:  '{"type":"final","answer":"Paris"}'
output: {"type":"final","answer":"Paris","evidence":[]}
```

Deliverable. One parsing module and tests for all four valid-input categories. Do not return the raw input string as an action.

Problem (protocol-invalid, 5 points)

Reject all of the following: empty text, `notjson`, `[]`, `null`, unknown type, blank tool name, array-valued args, numeric answer, object-valued evidence, negative or Boolean sentence indices, explanatory text after JSON, and two consecutive JSON objects. Error messages should identify the offending field where feasible. **Deliverable.** At least one negative test per case, all requiring `ValueError`. **Check.** Parsing failure must never trigger a tool call or a network request.

Problem (protocol-boundary, 2 points)

Why should valid JSON such as `"type":"tool_call","tool":"missing","args":{}` pass the parser but fail during execution? Explain the boundary in four to six sentences in your report.

Milestone: M1

You now have trustworthy action objects, even though no tools exist yet. Once the parser passes its tests, continue; the absence of a real model is not a blocker.

4 Retrieval: start with three documents (12 points)

Knowledge: Separate locating an article from reading it

`search(query,k)` finds candidate titles; `lookup(title)` returns their original sentences. This distinction enables two-hop behavior: consult the Ada Lovelace article to discover Lord Byron, then consult the Lord Byron article to determine his birthplace. Establish a simple deterministic baseline before introducing embeddings, vector databases, or web search.

4.1 Fixed toy environment and evidence indices

```
toy_context = [
 ["Ada Lovelace", [
  "Ada Lovelace worked on the Analytical Engine.",
  "Ada's father was Lord Byron."]],
 ["Lord Byron", [
  "Lord Byron was a British poet.",
  "He was born in London."]],
 ["Other Document", [
  "A paragraph about plants and gardening."]]
]
```

A `context` is a `list[[title,sentences]]`. The list index of each original sentence is the official evidence ID: Ada's father relationship is `["AdaLovelace",1]` and Byron's birthplace is `["LordByron",1]`. **Do not** resegment, reorder, deduplicate, or renumber sentences; otherwise evidence scoring will become inconsistent with the benchmark.

4.2 Minimal retrieval algorithm: token-set overlap

Knowledge: Learn retrieval through reproducibility before optimizing quality

Lowercase the query and each document's title plus all its sentences. Extract English alphanumeric tokens using `[a-z0-9]+` and discard duplicates. If Q is the query token set and D_i the set for document i, define

$$s(q, d_i) = |Q \cap D_i|.$$

Keep only documents with $s > 0$; order by **descending score** and then **ascending original title**, and take the first k. This is not BM25, and the ranking must never use gold answers or supporting facts.

`tokenize_text(text:str)->set[str]`

Purpose: Provide consistent tokenization for set-based retrieval.

Contract: Extract lowercase English alphanumeric tokens matching `[a-z0-9]+`; return a deduplicated set.

Failure/edge: Raise `ValueError` for non-string input; return an empty set for the empty string.

`run_search(context:list,query:str,k:int=3)->list[str]`

Purpose: Return only document titles ranked by relevance.

Contract: Use token-set intersection, descending score, and ascending original title. `k=0` or no matches returns `[]`.

Failure/edge: Raise `ValueError` for a non-string query or negative, Boolean, or non-integer `k`. Do not return full text or gold labels.

`run_lookup(context:list,title:str)->dict`

Purpose: Read a document by exact title while preserving original positions.

Contract: Use case-sensitive exact matching; return `"title":str,"sentences":list[str]` in the original sentence order.

Failure/edge: Raise `ValueError` for an unknown or non-string title. Never silently substitute a similar title.

Problem (retrieval-basic, 8 points)

1. Implement `tokenize_text` first; check that `"Britishpoet"` produces exactly two tokens.
2. Implement an internal helper to compute one document's overlap score; verify its output by hand on the toy context.
3. Implement ranking and top-k; verify that `run_search(toy_context,"AnalyticalEngine",2)` returns `["AdaLovelace"]`.
4. Implement `run_lookup(toy_context,"AdaLovelace")` and verify both original sentences are returned unchanged.

Deliverable. Test ordinary queries, blank queries, no matches, score ties, `k=0`, negative and Boolean `k`, and unknown titles.

Problem (retrieval-proof, 4 points)

In your report, explain why this baseline uses sets instead of word-frequency counts, what two distinct failure modes are observable when search and lookup are separate, and why gold answers must not affect ranking. Include a score-tie example of your own.

Milestone: M2

Without calling an LLM, you can manually execute `search("Ada...")`, `lookup("AdaLovelace")`, and `lookup("LordByron")` in a Python REPL. You have built pure retrieval functions, not yet an agent.

5 Tool executor: controlled action execution (10 points)

Knowledge: A model request is not program authorization

A tool registry is $R = \{\text{name} \mapsto \text{callable}\}$. The controller may select only explicitly registered functions. Never resolve a model-supplied name through `globals()`, arbitrary module reflection, or a shell. Bind action arguments as keyword arguments to an authorized callable. A tool exception must become a *failure observation readable by the model*, not an unexpected crash of the entire agent loop.

`execute_tool(action:dict,tools:dict,max_observation_chars:int=8000)->dict`

Purpose: Validate execution requirements, call a registered function, and format observations consistently.

Contract: Return `"ok":true,"observation":"..."` on success. Preserve strings; serialize lists/dictionaries to deterministic JSON. Document a consistent policy for other return values.

Failure/edge: For unknown tools, invalid arguments, tool exceptions, or excessive outputs, return

`ok=false` with a string beginning `ERROR:`. Do not execute another function instead.

`make_tools(context:list)->dict`

Purpose: Create a registry bound to *one question's* document context.

Contract: Expose at least `search` and `lookup` callables. Bind `context` through a closure or object; the model supplies only `query,k` or `title`.

Failure/edge: Never expose previous questions' documents through the current registry.

A long observation must **never grow the history without limit**. Within a maximum of 8,000 characters, return a visibly truncated preview with an explicit truncation marker and `ok=false`. Every attempted tool call counts toward the tool budget, even when a name is unknown or arguments are invalid. Catch `Exception`, but do not suppress termination signals such as `KeyboardInterrupt`.

Problem (tool-registry, 6 points)

Start with an injected test registry such as `"echo":echo` instead of hard-coding search and lookup. In order, test: successful echo, unknown name, missing argument, surplus argument, tool-raised exception, dictionary/list return, and oversized string output. **Deliverable.** A reusable execution adapter and at least seven tests.

Problem (context-binding, 4 points)

Create two disjoint toy contexts A and B and call `make_tools` separately on each. Prove that a lookup through A's registry cannot read a title occurring only in B. **Deliverable.** Two registries, an isolation test, and an explanation of why closure binding is safer than allowing the model to pass `context` itself.

Milestone: M3

You can now manually construct a valid action dictionary and pass it through `execute_tool` to obtain a structured observation. The next step adds model decisions and an iterative controller.

6 Messages, prompting, and the ReAct loop (20 points)

Knowledge: An agent is a finite-state machine with explicit budgets

The state includes at least h_t (messages), c_m (completed *logical model calls*), c_t (attempted tool calls), and the trace. Each model request increases $c_m \leftarrow c_m + 1$. When the model returns a tool call and sufficient budget remains, increase $c_t \leftarrow c_t + 1$. Failed tools also count. A final action ends successfully; invalid model output or a model exception leads to `error`; an exhausted model-step budget leads to `max_steps`; requesting another tool when no tool budget remains leads to `tool_limit`. These counters measure different events and must not be conflated.

6.1 Message format and visible history

```
initial messages:
[
 {"role":"system", "content":"<protocol + tool instructions>"},
```

```
 {"role":"user", "content":"Where was Ada Lovelace's father born?"}
]

after the model requests lookup:
 {"role":"assistant", "content":"{\"type\":\"tool_call\",...}"}
 {"role":"user", "content":"Observation: {\"title\":\"Ada Lovelace\",...}"}
```

A model sees only the `messages` supplied to the current request. Printing a tool result to the console or writing it to a trace file does not place the observation in the model's context. In this assignment, observations are wrapped in `user` messages with an explicit prefix; **native API tool-call roles are not used**. Mark tool output as external data so it cannot supersede system-level rules.

`build_initial_messages(question:str,tool_descriptions:str,max_steps:int,max_tool_calls:int)->list[dict]`

Purpose: Create the initial system and user messages.

Contract: The system message explains the single-JSON-object protocol, allowed tools and their arguments, evidence references, and budgets; the user message carries the question.

Failure/edge: Do not insert the complete question context, gold answer, or supporting facts. Reject invalid parameters explicitly.

`append_observation(messages:list[dict],raw_action:str,observation:str)->list[dict]`

Purpose: Append a model action and the actual tool observation to the next-round history.

Contract: Return a new list containing the original messages followed by an assistant raw-action message and a user observation with a clear `Observation:` prefix.

Failure/edge: Do not drop the original question or promote untrusted observation text to a system instruction.

`run_agent(question:str,model,tools:dict,max_steps:int=8,max_tool_calls:int=6)->dict`

Purpose: Run the full agent decision loop using a backend-agnostic interface.

Contract: The model implements `generate(messages)->str`. At a minimum, return `status,answer,evidence,trace,num_llm_calls,num_tool_calls`.

Failure/edge: For non-final outcomes, use `answer=""` and `evidence=[]`. Raise `ValueError` for invalid budgets before any model call; stop in a controlled way on all other failures.

6.2 State-transition table and important boundary cases

Model decision	Status	Required controller behavior
Valid final	`final`	Record answer and evidence; never call a tool or model again
Valid tool call, budget left	Continue	Attempt tool, append observation, record trace
Valid tool call, no budget	`tool_limit`	Do not execute tool or increment tool attempts
Parse error / generate exception	`error`	Record raw output and error; do not treat it as an answer
Tool call at the final model step	`max_steps`	Never silently make an extra model request
`max_steps=0`	`max_steps`	Terminate immediately with zero model requests

Important counterexample: `max_steps=3,max_tool_calls=2` must permit *tool call → tool call → final*. Exhausting the tool budget does not mean the controller must immediately stop: it stops with `tool_limit` only if a *subsequent action still requests a tool*. Conversely, when `max_steps=2,max_tool_calls=2`, a tool call at step two must not trigger a secret third model request.

6.3 Isolate controller bugs with a scripted model

Write the smallest possible `ScriptedModel` in your tests. Its constructor receives a list of predetermined outputs; `generate(messages)` returns them in order and saves every received message list. Test the following two model responses:

```
{"type":"tool_call","tool":"lookup",
 "args":{"title":"Ada Lovelace"}}
{"type":"final","answer":"Lord Byron",
 "evidence":[["Ada Lovelace",1]]}
```

Required two-hop extension. For the question *Where was Ada Lovelace's father born?*, construct a separate three-step scripted output sequence: *lookup Ada Lovelace → lookup Lord Byron → final London*. The final evidence must contain both citations. The agent must use three `generate` calls and two tool calls; it must not skip the second lookup because the test happens to know the answer. This test checks **continuous propagation of observations** required for multi-hop reasoning.

Problem (prompt-and-history, 4 points)

Test `build_initial_messages` and `append_observation` independently. Verify that the model's second request still contains the initial question and the observation mentioning `LordByron`; tool-supplied text must not change the original system instructions. **Deliverable.** Assertable snapshots of the complete message lists.

Problem (agent-happy-path, 8 points)

Implement the shortest complete controller using only ScriptedModel and the toy context. Proceed in this order: (1) build messages; (2) generate; (3) parse; (4) either return final or execute a tool; (5) append the observation; (6) repeat. **Deliverable.** A `status="final"` result with `num_llm_calls=2` and `num_tool_calls=1`, where the second model request contains the first tool observation.

Problem (loop-budgets, 5 points)

Test `max_steps=0`, `max_tool_calls=0`, negative or Boolean budgets, *tool/tool/final*, *tool/tool/tool*, and *tool at last step*. **Deliverable.** Correct statuses, counts, and actual tool side-effect counts for every path; failed tools must consume an attempted call.

Problem (loop-failure-recovery, 3 points)

Script a tool error followed by a model-generated correction and a final answer. Separately generate invalid JSON and a model exception. **Deliverable.** A tool error appears as an observation, allowing correction, whereas a protocol or model error terminates in a controlled state while retaining the trace. Automatic JSON repair is not required for the base

assignment.

Milestone: M4 — the most important checkpoint

At this point you have a **complete ReAct agent controller**; only the decision maker is simulated by ScriptedModel. We strongly recommend **not** connecting a paid model API before this checkpoint passes. When it does, integrating a real model amounts to replacing the `model` object.

7 Provider adapters: DeepSeek and Ollama (10 points)

Knowledge: Transport, action parsing, and control are distinct interfaces

A provider's SDK response may contain `choices`, `message`, `content`, usage, or reasoning-related fields. The sole responsibility of `generate` is to convert that response into a **string containing the assistant's visible response**. `parse_action` converts the string to a validated action, and `run_agent` decides whether it can be executed and when to stop. Swapping providers must not change the protocol, tools, or controller.

`make_model(backend:str,model_id:str,base_url:str,api_key:str|None=None,timeout:float=60.0)`

Purpose: Construct a model adapter exposing `generate(messages)->str`.

Contract: Support `deepseek` and `ollama`; pass the base URL explicitly; use a finite timeout; return only the assistant's visible text for logical generation.

Failure/edge: Missing configuration, connection or authorization errors, missing content, and empty responses must produce controlled exceptions. Never disguise failure as an empty final answer.

Configuration	DeepSeek	Ollama
`REACT_BACKEND`	`deepseek`	`ollama`
`REACT_BASE_URL`	`https://api.deepseek.com`	`http://localhost:11434/v1`
`REACT_MODEL`	Configurable, valid official model ID	`qwen2:7b` (or installed ID)
`DEEPSEEK_API_KEY`	Read locally from `.env`; never submit	No cloud key required. An SDK may need a placeholder
`REACT_TIMEOUT`	Finite, e.g. 60 seconds	Finite, e.g. 60 seconds

Local setup. Install Ollama, run `ollamapullqwen2:7b`, and verify its actual model ID using `ollamalist`. Use `ollamaserve` if the service is not running. Either the native `/api/chat` or the OpenAI-compatible `/v1` interface is acceptable, but record which one you used. **Cloud setup.** Obtain a DeepSeek API key and consult the official Chat Completions documentation for valid model IDs at execution time rather than hard-coding an old alias.

Knowledge: HTTP attempts are not the same as logical decision steps

One `generate` call may retry a transient HTTP failure a limited number of times, but it still counts as *one logical agent decision step*. With `max_steps=8` and up to three HTTP attempts per step, a run could issue as many as 24 network requests. Configure the SDK

retry behavior (or implement a bounded policy), record `num_http_attempts` where observable (or use null when unknown), and never retry authentication errors indefinitely.

Problem (provider-contract, 4 points)

First use a fake successful SDK response to test content extraction without network access. Then simulate `content=None`, missing choices, whitespace-only content, timeouts, and authentication failures. **Deliverable.** Two backend construction paths and an explicit exception policy, both presenting the same `generate(messages)->str` contract.

Problem (provider-live, 4 points)

Use **one** backend to complete (a) a connectivity request and (b) at least one real agent run over the toy context. You must implement the configuration entry point for the other backend, but need not pay for or download a second model solely to run this test. **Deliverable.** `reports/backend.md` with the backend, model ID, base URL, timestamp, real/simulated designation, latency, issues, and fixes. **Never** record keys.

Problem (provider-boundaries, 2 points)

Contrast "the model returned non-JSON text" with "the HTTP request timed out." Which layer handles each event? Should either be retried? Explain why neither may be misrepresented as `status=final`.

Milestone: M5

You now have a **real end-to-end ReAct agent**. If it frequently violates the JSON contract, inspect the prompt and raw outputs first. Do not weaken the parser to accept arbitrary prose simply to make one demonstration succeed.

8 Trace, replay, and experimental integrity (6 points)

Knowledge: Debugging traces differ from model-visible context

`messages` contain only what the model can see; a `trace` should additionally record the decision index, raw generation, parsed action, tool status, errors, elapsed time, and remaining budgets. This lets you identify the *first* deviation. JSONL is useful for batch processing: store **one complete question run per line**. It supports interruption and resume and makes individual failures easy to inspect.

```
{
 "sample_id":"toy-001", "run_id":"react-smoke-v1",
 "status":"final", "answer":"Lord Byron",
 "evidence":[["Ada Lovelace",1]],
 "num_llm_calls":2, "num_tool_calls":1,
 "elapsed_seconds":null,
 "config":{"backend":"scripted", "prompt_version":"v1"},
 "trace":[
  {"step_index":0,"raw_output":"...",
   "action":{"type":"tool_call","tool":"lookup"},
```

```
    "tool_result":{"ok":true,"observation":"..."},
    "error":null,"elapsed_seconds":null},
   {"step_index":1,"raw_output":"...",
    "action":{"type":"final","answer":"Lord Byron"},
    "tool_result":null,"error":null,"elapsed_seconds":null}
  ]
 }
```

In this illustration, null values and ellipses stand for example fields, not measured timings or actual complete generations. Real trace entries must be complete enough to reconstruct both decisions.

`write_jsonl_record(path,record:dict)->None`

Purpose: Append a complete question-level record to a run log.

Contract: Use UTF-8; each record must occupy exactly one valid JSON line and contain the sample ID, run ID, outcome, trace, and configuration reference.

Failure/edge: Do not record API keys, Authorization headers, or private reasoning. Diagnose a truncated final line explicitly instead of silently treating it as success.

`load_completed_ids(path,run_id:str)->set[str]`

Purpose: Find question IDs already fully recorded under a particular run ID before resuming.

Contract: Count only parseable, complete records whose run ID matches; use the returned set to skip work already finished.

Failure/edge: A partial JSON line or a different run ID must not cause an incorrect skip. Flag duplicate records explicitly.

Problem (trace-schema, 4 points)

Write JSONL examples for a successful final answer, a tool error followed by correction, and a budget-exhaustion case. Every generation step must include `step_index` (zero-based), `raw_output`, `action`, `tool_result`, `error`, and `elapsed_seconds`. Global call counts must be reconstructible from the steps. **Deliverable.** Actual log records and a short field glossary.

Problem (trace-resume, 2 points)

Write results for two toy questions, simulate an interruption, and resume to run only the remaining IDs. A second resume must not append duplicates. Specify an explicit failure or repair strategy for a corrupted partial line. **Deliverable.** A successful resume demonstration and an explanation of why temperature zero does not guarantee byte-for-byte determinism from a cloud model.

9 HotpotQA: from toy documents to real data (6 points)

Knowledge: Candidate-document QA is not a full-Wikipedia search

Every HotpotQA development-distractor example provides a `question`, a candidate-document `context`, an `answer`, and `supporting_facts`. The agent may use search and lookup **only over the candidate documents supplied for that question**: this is a *closed-candidate retrieval* task. It is *not* HotpotQA's fullwiki leaderboard setting, nor open internet search. Examples include bridge and comparison reasoning types, which can be

> grouped during offline analysis.

```
{
 "_id":"example-id",
 "question":"Where was Ada Lovelace's father born?",
 "context":[["Ada Lovelace",["...","..."]],
            ["Lord Byron",["...","..."]]],
 "answer":"London",
 "supporting_facts":[["Ada Lovelace",1],["Lord Byron",1]],
 "type":"bridge", "level":"easy"
}
```

This is an invented example of the data structure, not a specific sample from the official dataset.

9.1 Prevent gold-label leakage

Field	Permitted access
`_id`, `question`	Run entry point and initial user question
`context`	Internal search/lookup closures for *this question only*
`answer`, `supporting_facts`	Offline scorer only; never a model input or tool result
`type`, `level`	Offline breakdowns only; no oracle routing during inference

`load_hotpot(path)->list[dict]`

Purpose: Load the HotpotQA development-distractor JSON and validate its basic structure.

Contract: Every `_id` is nonempty and globally unique; `question` is a string; `context` contains `[title,sentences]` items, with sentences left in original order.

Failure/edge: Explicitly report missing fields, duplicate IDs, duplicate titles within a question, and invalid types. Do not silently discard examples.

`split_example(example:dict)->tuple[dict,dict]`

Purpose: Physically separate runtime input from offline gold annotations.

Contract: The first dictionary contains only ID, question, and context; the second contains only ID, answer, supporting facts, and metadata needed for offline aggregation.

Failure/edge: Do not hand a full, unsanitized example to the prompting function on the assumption that the answer will be removed later.

Problem (dataset-loader, 2 points)

Download the development-distractor data described in the official HotpotQA repository. Load and validate it without calling a model; inspect the sentence indices of one or two articles in a sampled question. **Deliverable.** Dataset file SHA256, loaded-example count, and format-validation logs. Do not paste the full dataset into your report.

Problem (data-isolation, 3 points)

Call `split_example` and `make_tools` so that the agent entry point receives only the question and a per-question registry. Before the first tool call, verify that messages contain **neither** the full candidate context **nor** gold answers or supporting facts. Check that registries from

two questions cannot access one another's documents. **Deliverable.** Two automatically runnable isolation tests.

Problem (evidence-contract, 1 point)

Final evidence must contain the *original article title and original, zero-based sentence index*, not search rank or candidate-document position. If the model cites a nonexistent sentence, never repair it using gold labels; preserve the prediction, flag the invalid citation in the trace, and let the offline scorer handle it. **Deliverable.** One single-question trace from real HotpotQA data.

10 Metrics and batch evaluation (12 points)

Knowledge: Answer correctness and evidence correctness require separate metrics

Answer Exact Match (EM) compares normalized strings. Answer F1 measures token overlap, treating **repeated words according to their multiplicity**, so a `Counter` is useful. If C is the common-token count, P the number of predicted tokens, and G the gold-token count, then

$$\text{precision} = \frac{C}{P}, \quad \text{recall} = \frac{C}{G}, \quad F_1 = \frac{2\,\text{precision}\,\text{recall}}{\text{precision} + \text{recall}}.$$

Use the official evaluator's conventions for zero denominators. Mismatched special answers among `yes/no/noanswer` must receive F1 zero. Supporting-fact (SP) precision, recall, F1, and EM are computed over the *set* of $(title, sent_id)$ tuples; predicting a sentence twice earns no extra credit. The official joint metrics combine answer and evidence correctness and are *not* simply the arithmetic mean of their F1 scores.

`normalize_answer(answer:str)->str`

Purpose: Implement the official HotpotQA English answer normalization.

Contract: In order: lowercase, remove `string.punctuation`, remove standalone articles a/an/the, and collapse whitespace.

Failure/edge: Raise `ValueError` for non-string input. Do not add synonyms or stemming.

`score_answer(prediction:str,gold:str)->dict`

Purpose: Compute answer EM, F1, precision, and recall for one question.

Contract: Return all four fields with numeric values between 0 and 1; count repeated tokens as a multiset and handle special answers according to the official evaluator.

Failure/edge: Follow the official `exact_match_score` and `f1_score` behavior for empty strings: when both are empty, EM=1 and F1=0.

`score_support(predicted:list,gold:list)->dict`

Purpose: Compute supporting-fact metrics over title/index pairs.

Contract: Deduplicate evidence; ignore citation ordering; return EM, F1, precision, and recall.

Failure/edge: When denominators are zero, P/R/F1=0; when both sets are empty, EM=1. Titles are matched exactly without answer-text normalization.

Problem (metric-unit-tests, 5 points)

Check that `normalize_answer("TheQUICK,BrownFox!")=="quickbrownfox"`. For answer `"AdaByron"` against gold `"AdaLovelaceByron"`, verify $P = 1, R = 2/3, F_1 = 0.8, EM = 0$. Also test `"redredblue"` versus `"redblueblue"` to check duplicate counts, as well as yes/no, empty strings, duplicate evidence, and evidence order. **Deliverable.** Agreement between your local scorer and the official evaluation script on the test examples.

10.1 Fixed splits: smoke, then development, then evaluation

Given the unique `_id` values in the complete official development-distractor dataset, construct the stable hash key

$$k(id) = \text{SHA256}(\text{UTF8}(\texttt{"42:"} + id)).\text{hexdigest}().$$

Sort by hash ascending (breaking hash ties by ascending ID string). The base assignment takes disjoint **20/50/200**-example smoke/development/evaluation slices at `[0:20]`, `[20:70]`, and `[70:270]`. This is an *assignment-specific protocol*, not an official hidden 200-question test set. Do not use Python's process-randomized `hash()`. **`make_fixed_splits(ids:list[str],seed:int=42)->dict[str,list[str]]`**

Purpose: Deterministically produce three disjoint lists of example IDs.

Contract: Return `smoke`, `dev`, and `eval` lists of sizes 20, 50, and 200 in the base configuration, identical across repeated processes.

Failure/edge: Reject duplicate IDs, fewer than 270 valid IDs, and invalid IDs. Never choose easy examples after inspecting model outcomes.

Problem (fixed-splits, 3 points)

Create `reports/splits/smoke_ids.txt` (20), `dev_ids.txt` (50), and `eval_ids.txt` (200), with one original ID per line. **Deliverable.** All three files, a dataset digest, and a disjointness test.

10.2 Batch execution and the official prediction format

```
{
 "answer": {"example-id": "London"},
 "sp": {"example-id": [
  ["Ada Lovelace",1],["Lord Byron",1]
 ]}
}
```

`make_predictions_json(records:list[dict],expected_ids:list[str])->dict`

Purpose: Convert terminal agent records into the official `answer`/`sp` prediction structure.

Contract: Every planned ID appears in both mappings. Use an empty answer and empty evidence for non-final outcomes; the key sets must match `expected_ids` exactly.

Failure/edge: Explicitly report duplicate IDs, missing IDs, or predictions from other splits. Never export only correctly answered questions.

`run_batch(data_path,ids_path,model_factory,method:str,config:dict,output_dir)->dict`

Purpose: Run independent per-question agents over one fixed split, saving predictions, traces, and aggregate metrics.

Contract: Support at least `method=react`; bind context separately for every example; count failures in the denominator; preserve actual call counts.

Failure/edge: Support deduplicated resume; never leak gold labels to the model; never label a partial run as a full evaluation.

Problem (batch-and-official-score, 4 points)

Run 20 smoke examples, then 50 development examples, and finally **freeze all settings** before running the 200-example evaluation. Each run directory must contain at least `config.json`, `ids.txt`, `predictions.json`, `metrics.json`, and `traces.jsonl`. Construct a *gold subset matching exactly the selected IDs* from the official source data for use *only by the scorer*. If the official script requires `ujson`, install it in the evaluator's environment rather than changing the scoring rules.

```
# Example commands you must make executable yourself
uv run python scripts/make_splits.py --data data/hotpot_dev_distractor_v1.json
uv run python scripts/evaluate.py --method react --backend ollama \
    --ids reports/splits/smoke_ids.txt --output-dir reports/runs/react-smoke
# Official scorer's CLI shape:
python hotpot_evaluate_v1.py predictions.json subset_gold.json
```

Deliverable. Answer EM/F1, SP EM/F1, Joint EM/F1, mean model/tool calls, error rate, invalid-action rate, and average time per question. Include every planned example in the denominator; cross-check with the official evaluator.

Milestone: Honest reporting with limited resources

You may start with **five diagnostic questions plus the 20-question smoke set** to verify the full pipeline. If model or compute budget is limited, report the *actual subset and sample count* rather than claiming a 200-question evaluation. Such results are not comparable to the official leaderboard. Without a working model, offline tests through M4 and M7 are still valuable.

11 Controls, ablation, and error analysis (6 points)

Knowledge: A controlled experiment asks about mechanisms, not prompt length

To test whether interactive retrieval is useful, compare three methods using the **same model, questions, and scorer**. The number of calls is not incidental: ReAct may improve accuracy while also increasing latency and cost. Report performance, successful completions, and resource consumption. In an ablation, change one variable at a time; do not repeatedly tune on the final evaluation split.

Method	Strict definition	Model calls
Direct	Question only; model gives final JSON, with no tools or context	1
Single Retrieval	Program runs fixed `search(k=1)` on the full question, then `lookup`; model reads result and emits final once	1
ReAct	Model chooses search / lookup / final, within eight decision steps and six tool calls	At most 8

Problem (baselines, 2 points)

Hold model ID, temperature, split, and retrieval algorithm fixed. Debug all three methods on development IDs, then compare on the same evaluation IDs. Report at least answer EM/F1, SP F1, Joint F1, model/tool calls, and latency. **Deliverable.** One table with actual measurements; label a dev-only experiment clearly if resources are insufficient.

Problem (ablation, 2 points)

Choose one variable, such as `max_tool_calls=6` versus `max_tool_calls=1`, and compare on fixed development IDs. Write a prediction *before* running the experiment, then explain the actual outcome, even if performance does not improve. **Deliverable.** A controlled comparison table and two paragraphs discussing mechanisms.

Problem (failure-cases, 2 points)

Analyze at least five failed examples (or all observed failures supplemented by controlled failure-injection tests). For each, provide sample ID, first consequential deviation, raw action, observation, predicted answer, offline gold answer, failure category, and a testable proposed fix. Suggested categories are initial retrieval failure, missing second hop, invalid tool arguments, malformed output, exhausted budgets, wrong evidence index, truncation, and misreading an observation. **Deliverable.** Trace-backed analysis, not a vague claim that "the model is weak."

12 Submission checklist and debugging guide

12.1 Final submission

1. **Code.** Your implementations under `react_agent/`, thin `adapters.py`, tests, scripts, `pyproject.toml`, `README.md`, and `.env.example`. Exclude the real `.env`, secrets, `.venv`, and full downloaded datasets.
2. **Writeup.** `reports/writeup.md` and your exported `writeup.pdf`. Include all written answers, environment, commands, model/configuration, actual results, ablations, and failure analysis.
3. **Experiments.** The three frozen ID splits, real run folders, at least three complete traces, backend connectivity record, and reproducibility metadata.

12.2 Acceptance checklist

☐ The parser validates only syntax and structure; invalid actions never invoke Python tools.

☐ The registry executes only registered functions; errors become observations and count as

attempts.

- ☐ ScriptedModel completes the toy "lookup Ada, lookup Byron, answer London" trajectory.
- ☐ The next `generate` request actually contains the previous observation.
- ☐ Tool / tool / final succeeds under three model steps and two tool calls.
- ☐ DeepSeek and Ollama share a model interface; at least one real backend was tested.
- ☐ Initial model messages do not contain full context, answers, or supporting facts; environments are isolated across questions.
- ☐ Original article titles and sentence arrays retain their zero-based indices.
- ☐ The 20/50/200 splits are deterministic and disjoint; non-final cases stay in the denominator.
- ☐ The official evaluator accepts the predictions; model ID, date, data hash, and config are documented.

12.3 Where to investigate common failures first

Symptom	First diagnostic check
Model produces beautiful prose but nothing executes	Is raw output *exactly* one JSON object? Inspect the prompt.
Model cannot use the previous document	Inspect next-round `seen_messages`, not only console output.
Agent crashes on the second tool attempt	Distinguish `max_steps` from `max_tool_calls`, including check order.
Local Ollama requests reach a remote endpoint	Inspect explicit base URL and SDK defaults.
Evidence indices are consistently wrong	Check resegmentation, reordering, and 0-based versus 1-based conversion.
EM/F1 is implausibly high or low	Check gold leakage, empty predictions, denominator, and evaluator ID sets.
Runs cannot be reproduced	Check split, sorting, and version drift before blaming model randomness.

13 Optional extensions (outside the base 100 points)

Only attempt these after passing the full basic pipeline. For each extension, use fixed comparisons and measured outcomes: **(i)** implement BM25 yourself and compare it with token overlap; **(ii)** compress/deduplicate message history and measure loss of evidence localization; **(iii)** compare OpenAI-compatible native tool calling with the JSON-in-text protocol for format compliance; **(iv)** implement model-driven self-correction, counting each new decision as a logical step; and **(v)** move to an interactive environment such as ALFWorld or WebShop, redefining actions, observations, and scoring. Never compare numeric scores across incompatible benchmarks as though they measure the same task.

A Interface reference: all public functions

This appendix indexes the public interfaces; **precise signatures, inputs, outputs, and error semantics appear in the corresponding main sections**. Private helpers not included here are yours to design. Follow the order below to ensure each finished function connects naturally to the next component.

Sec.	Function	Main responsibility
2	`parse_action`	JSON syntax and action-structure validation
3	`tokenize_text`	Regex-based token sets
3	`run_search`	Candidate article-title retrieval
3	`run_lookup`	Original sentence-list lookup
4	`execute_tool`	Authorized tool execution and observation formatting
4	`make_tools`	Bind a question-specific data source
5	`build_initial_messages`	Prompt, roles, and action protocol
5	`append_observation`	Maintain model-visible history
5	`run_agent`	Iterative control, termination status, and counts
6	`make_model`	Common backend interface
7	`write_jsonl_record`	Persist question-level traces
7	`load_completed_ids`	Resume and deduplication
8	`load_hotpot`	Dataset loading and validation
8	`split_example`	Separate runtime inputs from labels
9	`normalize_answer`	Normalize gold/predicted answers
9	`score_answer`	Answer EM / F1 / precision / recall
9	`score_support`	Supporting-fact metrics
9	`make_fixed_splits`	Reproducible data partitioning
9	`make_predictions_json`	Official evaluator JSON format
9	`run_batch`	Independent batched runs and aggregation

B Test matrix: what to test before writing the full agent

Suggested test file	High-priority assertions	Prevented failure mode
`test_protocol.py`	Valid final/tool; invalid JSON/evidence	Treating invalid text as actions
`test_retrieval.py`	Query/k boundaries, sorting, unchanged sentences	Unstable retrieval or citation drift
`test_tools.py`	Echo/unknown/raise/long/context A–B	Unauthorized execution or crashes
`test_loop.py`	Two-step success, three-step boundary, states and counts	Tool output never becomes model input
`test_dataset.py`	Example fields, no leakage, isolation	Gold labels entering prompt
`test_metrics.py`	Duplicate token counts, yes/no, empty, SP deduplication	Divergence from official scorer

C Reference materials

1. Stanford CS336 Staff. *Assignment 1 (basics): Building a Transformer LM*, version 26.0.3 (2026). This handout borrows its pedagogical organization: problems, deliverables, from-scratch work, adapters, tests, and resource-conscious development.
   CS336 Assignment 1 repository.

2. Yao, S. et al. *ReAct: Synergizing Reasoning and Acting in Language Models.* arXiv:2210.03629 (2022; v3, 2023).
   ReAct paper on arXiv.

3. Official HotpotQA repository. Describes development-distractor data, `context`, `supporting_facts`, and prediction formatting.
   HotpotQA repository.

4. Official HotpotQA evaluator for answer, evidence, and joint scoring.
   HotpotQA evaluation script.

5. DeepSeek Chat Completions API. Verify model IDs and response fields.
   Official API documentation.

6. Ollama: OpenAI compatibility documentation; Qwen2 model library.

7. uv documentation. Isolated environments, dependency management, and test execution.
   Official uv manual.

Document note. This PDF is not a bundled code repository: you must create and implement the listed tests, scripts, and functions yourself. Example traces, timings, and scores do not represent actual experimental results. Never modify gold labels or shrink a frozen evaluation split to make tests pass.